

Analisis Metodologia de la Programacion

Producto final del programa

En metodologia de la programacion se puede justificar el proyecto como un programa ejecutable en Python que evalua el estado de seguridad de un sistema frente a actividad sospechosa.

El producto no pide los datos manualmente con input, porque el proyecto trabaja como simulador. En lugar de que el usuario escriba los valores, el programa genera datos automaticamente para simular una red durante 60 segundos.

Entrada simulada -> procesamiento -> estado de seguridad -> alertas, graficas y CSV

Archivo principal ejecutable: m00_pdi_slp.py

Salida principal del sistema:

```
| Salida | Significado |
|---|---|
| NORMAL | seguridad estable |
| SOSPECHOSO | posible comportamiento irregular |
| ALERTA | riesgo detectado |
| CRITICO | riesgo alto y protecciones reforzadas |
```

El programa tambien muestra mensajes en consola, genera graficas y crea un reporte CSV.

Entradas del programa

El documento de metodologia pide datos como uso de CPU, intentos fallidos, archivos desconocidos y mensajes sospechosos. En este proyecto esos datos se representan con variables del paquete.

Variables de entrada usadas:

```
| Dato pedido | Variable usada en el programa | Tipo de dato |
|---|---|---|
| uso de CPU | cpu y cpu_base | float / int |
| intentos fallidos de acceso | intentos_login | int |
| computadoras con fallos | se representa como estado del sistema y central monitoreada | string |
| archivos desconocidos | cambios_archivos | int |
| mensaje sospechoso | incidente y tipo_incidente | bool / string |
| salida de datos | salida_datos | int |
| usuario | usuario | string |
| IP | ip | string |
```

```

paquete = {
    "segundo": segundo,
    "central": central,
    "usuario": usuario,
    "ip": ip,
    "paquetes": random.randint(80, 420),
    "intentos_login": random.randint(0, 2),
    "cambios_archivos": random.randint(0, 3),
    "salida_datos": random.randint(1, 12),
    "incidente": False,
    "tipo_incidente": "NINGUNO"
}

```

Este bloque representa las entradas del programa. Aunque no se escriben manualmente, cumplen la función de datos iniciales que el sistema necesita para evaluar la seguridad.

Procesamiento del programa

El procesamiento consiste en tomar los datos del paquete, calcular métricas y decidir el estado del sistema.

Datos del paquete -> cálculos -> reglas -> estado final del ciclo

```

flujo = lm.calcular_flujo_digital(
    paquete["paquetes"], paquete["intentos_login"],
    paquete["cambios_archivos"], paquete["salida_datos"]
)

```

Este bloque procesa los datos principales y los convierte en un índice de flujo digital. Ese valor ayuda a saber si la actividad parece normal o riesgosa.

```

est, alertas, protecciones, cifrado, refriger = lm.evaluar_estado(
    paquete, flujo, paquete["temperatura"], estado_actual["cifrado"],
    indice_oscilatorio
)

```

Aquí se evalúa el paquete. La función devuelve el estado del sistema, las alertas, las protecciones, el nivel de cifrado y si se debe activar refrigeración.

Condicionales y operadores lógicos

El programa usa condicionales para tomar decisiones. Esto cumple con el requisito de tener bloques if, elif, else y operadores logicos como and, or y not.

```
if paquete["paquetes"] > srv.U_PAQUETES_ALERTA and paquete["intentos_login"] > srv.U_LOGIN_ALERTA:
    alertas.append("TRAFICO MASIVO + FUERZA BRUTA")
    estado = "ALERTA"
```

Este bloque usa and. Significa que se necesita trafico alto y tambien muchos intentos de login para activar esa alerta.

```
if paquete["paquetes"] > srv.U_PAQUETES_CRIT or flujo > srv.U_FLUJO_CRITICO:
    estado = "CRITICO"
    cifrado = "REFORZADO"
```

Este bloque usa or. Significa que basta con que una de las dos condiciones sea grave para pasar a estado critico.

```
elif temperatura >= srv.U_TEMP_REFRIG and not refrigeracion:
    refrigeracion = True
```

Este bloque usa not. Sirve para activar refrigeracion preventiva solo si todavia no estaba activa.

Ciclos iterativos

El programa usa ciclos for, que cumplen con el requisito de ciclos iterativos.

```
for segundo in range(1, 61):
```

Este ciclo es el principal. Repite la simulacion 60 veces, una por cada segundo.

```
for a in agentes_norte:
    agentes_autorizados.append(a)
for a in agentes_centro:
    agentes_autorizados.append(a)
for a in agentes_sur:
    agentes_autorizados.append(a)
```

Estos ciclos juntan los agentes autorizados de cada central en una sola lista general.

```
for red in range(1, 4):
    for host in range(10, 18):
        ips_internas.append("192.168." + str(red) + "." + str(host))
```

Este ciclo doble genera IPs internas sin escribirlas una por una.

Listas usadas en el programa

El requisito pide al menos una lista con minimo 7 elementos. El programa cumple esto con varias listas.

Lista con mas de 7 elementos: ips_internas

```
ips_internas = []
for red in range(1, 4):
    for host in range(10, 18):
        ips_internas.append("192.168." + str(red) + "." + str(host))
```

Esta lista termina teniendo 24 IPs internas, porque se generan 3 redes y 8 hosts por red.

3 redes * 8 hosts = 24 IPs

Otras listas del programa:

Lista	Uso
centrales	centrales monitoreadas
agentes_norte	usuarios autorizados del norte
agentes_centro	usuarios autorizados del centro
agentes_sur	usuarios autorizados del sur
usuarios_sospechosos	usuarios externos sospechosos
ips_sospechosas	IPs usadas por ataques
tipos_expediente	tipos de archivos o expedientes
TIPOS_ATAQUE	ataques que puede inyectar el simulador

Salidas del programa

Las salidas son los mensajes, estados, recomendaciones y archivos generados.

```
print(" Estado operativo      : " + estado)
print(" Cifrado activo         : " + cifrado)
print(" Refrigeracion            : " + texto_refrigeracion)
```

Estos mensajes muestran el estado del sistema en consola.

```
if len(alertas) > 0:
    print("  -- ALERTAS OPERATIVAS --")
    for a in alertas:
        print("    [!] " + a)
```

Esta salida imprime las alertas detectadas.

```
if len(protecciones) > 0:
    print("  -- PROTECCIONES ACTIVADAS --")
    for p in protecciones:
        print("    [OK] " + p)
```

Esta salida imprime las protecciones que se activaron, como cifrado, refrigeracion o bloqueo de login.

Salidas finales:

```
| Salida | Archivo o lugar |
|---|---|
| monitoreo por consola | terminal |
| graficas | pdi_slp_graficas/ |
| reporte CSV | reportes_tabla/pdi_slp_reporte_tabla.csv |
| resumen final | consola |
```

Porcentaje de seguridad

El documento de metodologia menciona que la evaluacion podria expresarse como porcentaje. El codigo actual no imprime literalmente un porcentaje de seguridad, pero si maneja cuatro niveles equivalentes de seguridad.

```
| Estado | Interpretacion posible |
|---|---|
| NORMAL | seguridad alta |
| SOSPECHOSO | seguridad media |
| ALERTA | seguridad baja |
| CRITICO | seguridad critica |
```

Si se quisiera adaptarlo sin romper la logica, se podria presentar asi en el documento:

```
NORMAL -> 90% a 100%
SOSPECHOSO -> 70% a 89%
ALERTA -> 40% a 69%
```

CRITICO -> 0% a 39%

No es necesario cambiar el código si el proyecto se presenta por estados, porque los estados ya representan el nivel de seguridad.

Resultados y analisis

El programa cumple con la estructura básica de metodología de la programación. Tiene entradas, procesamiento y salidas. También usa variables de diferentes tipos, listas, diccionarios, ciclos, condicionales, operadores lógicos y funciones.

La principal adaptación es que el usuario no introduce los datos manualmente. En su lugar, el simulador genera automáticamente los valores, lo cual tiene sentido porque el proyecto representa monitoreo de red en tiempo real.

Conclusiones

Metodología de la programación se refleja en la organización del sistema: archivos separados, funciones, datos de entrada, procesamiento, salidas y estructuras de control.

El prototipo cumple como programa ejecutable en Python para evaluar seguridad digital. Aunque no usa input, si solicita datos de forma simulada mediante generación automática, lo que permite probar muchos escenarios sin capturarlos manualmente.

Si se quisiera adaptar más al formato exacto de la materia, se podría agregar una versión pequeña con input para capturar manualmente CPU, login, fallos y mensaje sospechoso. Pero para el proyecto completo conviene mantener la simulación automática porque ya analiza 60 segundos, genera reportes y produce gráficas.